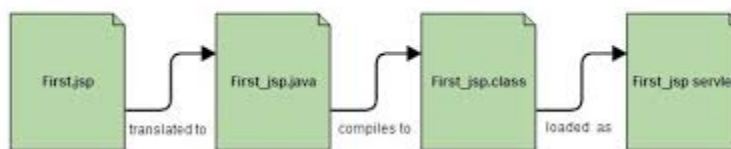
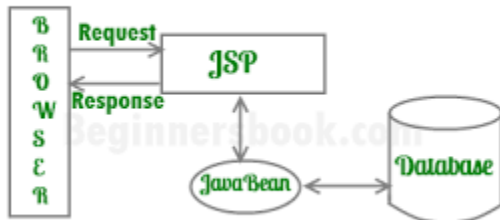


Introduction to JSP

JSP stands for **Java Server Pages**. It is a server-side technology which is used for creating web applications. It is used to create dynamic web content. JSP consists of both HTML tags and JSP tags. In this, JSP tags are used to insert JAVA code into HTML pages. It is an advanced version of **Servlet** Technology i.e. a web-based technology that helps us to create dynamic and platform-independent web pages. In this, Java code can be inserted in HTML/XML pages or both. JSP is first converted into a servlet by the JSP container before processing the client's request. JSP has various features like JSP Expressions, JSP tags, JSP Expression Language, etc.



How JSP more advantageous than Servlet?

- They are easy to maintain.
- No recompilation or redeployment is required.

- Less coding is required in JSP.
- JSP has access to the entire API of JAVA.
- JSP are extended version of Servlet.

Features of JSP

- **Coding in JSP is easy** : As it is just adding JAVA code to HTML/XML.
- **Reduction in the length of Code** : In JSP we use action tags, custom tags etc.
- **Connection to Database is easier** : It is easier to connect website to database and allows to read or write data easily to the database.
- **Make Interactive websites** : In this we can create dynamic web pages which helps user to interact in real time environment.
- **Portable, Powerful, flexible and easy to maintain** : as these are browser and server independent.
- **No Redeployment and No Re-Compilation** : It is dynamic, secure and platform independent so no need to re-compilation.
- **Extension to Servlet** : as it has all features of servlets, implicit objects and custom tags
 1. **Declaration Tag** : It is used to declare variables.
 2. **Java Scriptlets** : It allows us to add any number of JAVA code, variables and expressions.
 3. **JSP Expression** : It evaluates and convert the expression to a string.
 4. **JAVA Comments** : It contains the text that is added for information which has to be ignored.

- Create html page from where request will be sent to server eg try.html.
 - To handle to request of user next is to create .jsp file Eg. new.jsp
 - Create project folder structure.
 - Create XML file eg my.xml.
 - Create WAR file.
 - Start Tomcat
 - Run Application
5. It does not require advanced knowledge of JAVA
 6. It is capable of handling exceptions
 7. Easy to use and learn
 8. It contains tags which are easy to use and understand
 9. Implicit objects are there which reduces the length of code
 10. It is suitable for both JAVA and non JAVA programmer
 11. Difficult to debug for errors.
 12. First time access leads to wastage of time
 13. It's output is HTML which lacks features.

Creating a simple JSP Page

hello.JSP :

JSP simply puts Java inside HTML pages. You can take any existing HTML page and change its extension to “.jsp” instead of “.html”. In fact, this is the perfect exercise for your first JSP.

Take the HTML file you used in the previous exercise. change its extension from “.html” to “.jsp”. Now load the new file, with the “.jsp” extension, in your browser.

You will see the same output, but it will take longer! But only the first time. If you reload it again, it will load normally.

What is happening behind the scenes is that your JSP is being turned into a Java file, compiled, and loaded. This compilation only happens once, so after the first load, the file doesn't take long to load anymore. (But every time you change the JSP file, it will be re-compiled again.)

Of course, it is not very useful to just write HTML pages with a .jsp extension! We now proceed to see what makes JSP so useful.

Adding dynamic content via expressions:

As we saw in the previous section, any HTML file can be turned into a JSP file by changing its extension to .jsp . Of course , what makes JSP useful is the ability to embed Java. Put the following text in a file. jsp extension (let us call it hello.jsp) , place it in your JSP directory, and view it in a browser.

```
<HTML>
```

```
<BODY>
```

```
Hello! The time is now <%= new java.util.Date() %>
```

```
</BODY>
```

```
</HTML>
```

Notice that each time you reload the page in the browser, it comes up with the current time. The character sequence.

<%= and %> enclose Java expressions, which are evaluated at run time.

This is what makes it possible to use JSP to generate dynamic HTML pages that change in response to user actions or vary from user to user.

Explain JSP Elements:

We will learn about the various elements available in JSP with suitable examples. In JSP elements can be divided into 4 different types.

These are:

- Expression
- Scriptlets
- Directives
- Declarations

Expression:

We can use this tag to output any data on the generated page. These data are automatically converted to string and printed on the output stream.

Syntax:

JSP Expressions are : <%= "Anything" %>

NOTE : JSP Expressions start with Syntax of JSP Scriptlets are with <%=and ends with %>. Between these, you can put anything that will convert to the String and that will be displayed.

Example:

```
<%= "HelloWorld!" %>
```

Scriptlets:

In this tag we can insert any amount of valid java code and these codes are placed in the _jsp Service method by the JSP engine.

Syntax:

```
<%//java codes%>
```

NOTE : JSP Scriptlets begins with <% and ends %> . We can embed any amount of Java code in the JSP Scriptlets. JSP Engine places these codes in the _jspService() method.

Variables available to the JSP Scriptlets are:

- Request
- Response
- Session
- Out

Directives:

A JSP “directive” starts with <%@ characters. In the directives, we can import packages , and define error-handling pages or the session information of the JSP page.

Syntax:

```
<%@directive attribute="value"% >
```

- page
- include
- taglib

Declarations :

This tag is used for defining the functions and variables to be used in the JSP.

Syntax:

```
<%!
```

```
//java codes
```

```
%>
```

NOTE : JSP Declaratives begins with <%! and ends %> with We can embed any amount of java code in the JSP Declaratives. Variables and functions defined in the declaratives are class-level and can be used anywhere on the JSP page.

Example :

```
<%@page import="java.util.*"%>
```

```
<HTML>
```

```
<BODY>
```

```
<%!
```

```
Date the Date=new Date(); Date getDate()
```

```
{
```

```
System.out.println("In getDate() method"); return theDate;
```

```
}
```



```
%>
```

```
Hello! The time is now<%=getDate()%>
```

```
</BODY>
```

```
</HTML
```

Example of a JSP Web Page:

```
<HTML>
```

```
<HEAD>
```

```
<TITLE>A Web Page</TITLE>
```

```
</HEAD>
```

```
<BODY>
```

```
<%out.println("Hello there!");%>
```

```
</BODY>
```

```
</HTML>
```

Run a Simple JSP Page:

Step-1: Save the JSP file using “.jsp” extension (ex- “hello.jsp”)

Step-2: Start the server

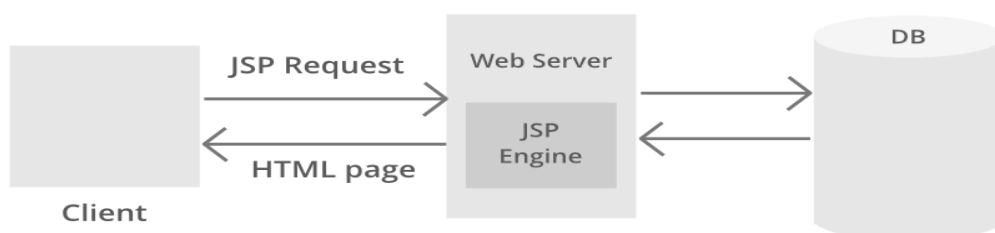
Step-3: Place your application inside a folder

Step-4: To execute the JSP script, simply start tomcat server and use a browser to browse an URL of the JSP page i.e.

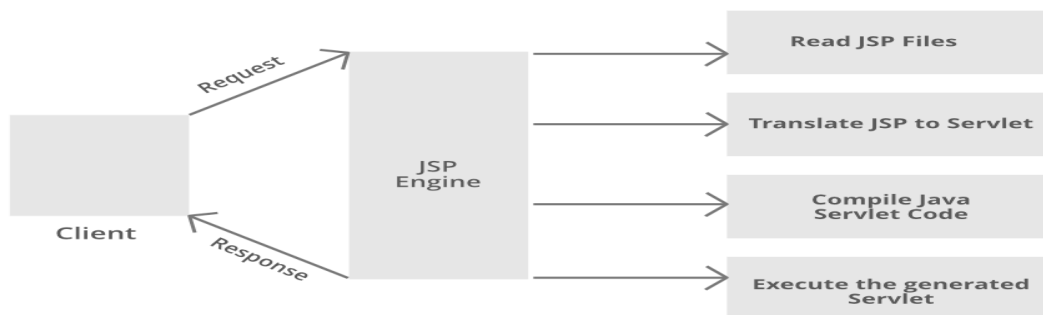
`http://localhost:portnumber/YourApplicationContextRoot/jspfile` then you will see the jsp file is being compiled.

SP Architecture

JSP architecture gives a high-level view of the working of JSP. JSP architecture is a 3 tier architecture. It has a Client, Web Server, and Database. The client is the web browser or application on the user side. Web Server uses a JSP Engine i.e; a container that processes JSP. For example, Apache Tomcat has a built-in JSP Engine. JSP Engine intercepts the request for JSP and provides the runtime environment for the understanding and processing of JSP files. It reads, parses, build Java Servlet, Compiles and Executes Java code, and returns the HTML page to the client. The webserver has access to the Database. The following diagram shows the architecture of JSP.



Now let us discuss JSP which stands for Java Server Pages. It is a server-side technology. It is used for creating web applications. It is used to create dynamic web content. In this JSP tags are used to insert JAVA code into HTML pages. It is an advanced version of Servlet Technology. It is a Web-based technology that helps us to create dynamic and platform-independent web pages. In this, Java code can be inserted in HTML/XML pages or both. JSP is first converted into a servlet by JSP container before processing the client's request. **JSP Processing** is illustrated and discussed in sequential steps prior to which a pictorial media is provided as a handful pick to understand the JSP processing better which is as follows:



Step 1: The client navigates to a file ending with the **.jsp extension** and the browser initiates an HTTP request to the webserver. For example, the user enters the login details and submits the button. The browser requests a status.jsp page from the webserver.

Step 2: If the compiled version of JSP exists in the web server, it returns the file. Otherwise, the request is forwarded to the JSP Engine. This is done by recognizing the URL ending with **.jsp extension**.

Step 3: The JSP Engine loads the JSP file and translates the JSP to Servlet(Java code). This is done by converting all the template text into `println()` statements and JSP elements to Java code. This process is called **translation**.

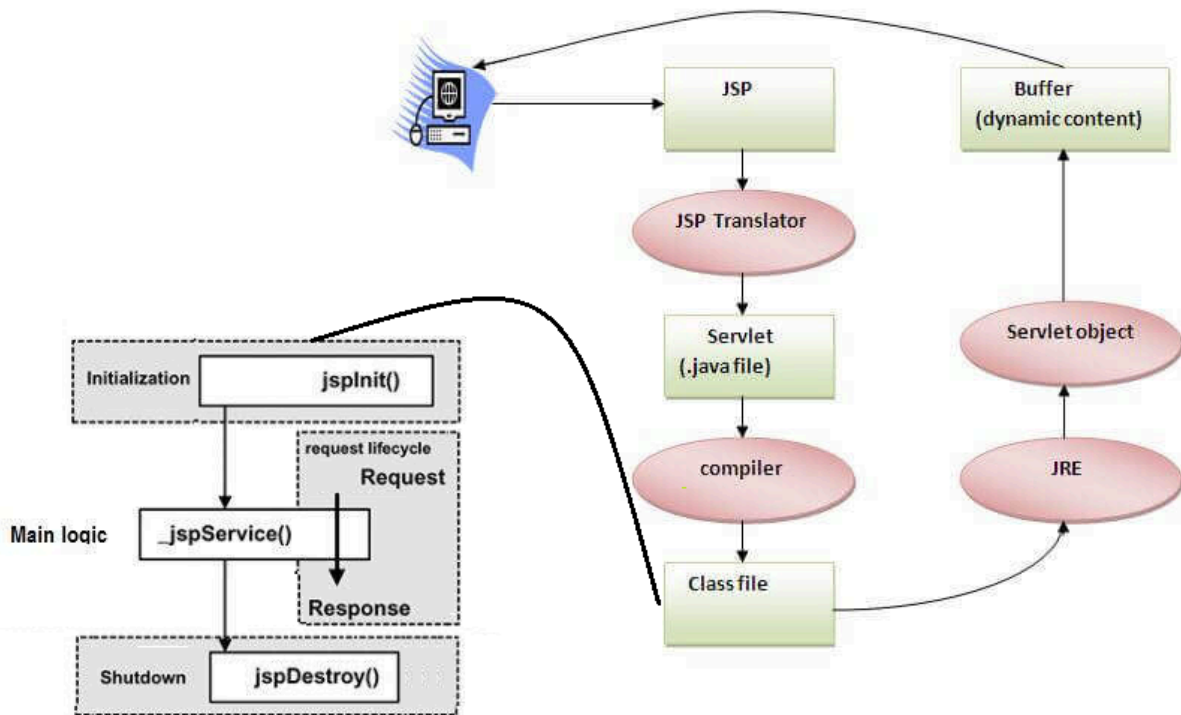
Step 4: The JSP engine compiles the Servlet to an executable **.class** file. It is forwarded to the Servlet engine. This process is called **compilation** or **request processing phase**.

Step 5: The **.class** file is executed by the Servlet engine which is a part of the Web Server. The output is an HTML file. The Servlet engine passes the output as an HTTP response to the webserver.

Step 6: The web server forwards the HTML file to the client's browser.

Life cycle of JSP

A Java Server Page life cycle is defined as the process that started with its creation which later translated to a servlet and afterward servlet lifecycle comes into play. This is how the process goes on until its destruction.



Lifecycle of JSP

Following steps are involved in the JSP life cycle:

1. Translation of JSP page to Servlet
2. Compilation of JSP page(Compilation of JSP into test.java)
3. Classloading (test.java to test.class)
4. Instantiation(Object of the generated Servlet is created)
5. Initialization(jspInit() method is invoked by the container)
6. Request processing(_jspService() is invoked by the container)
7. JSP Cleanup (jspDestroy() method is invoked by the container)

SP Implicit Objects

There are **9 jsp implicit objects**. These objects are *created by the web container* that are available to all the jsp pages.

The available implicit objects are out, request, config, session, application etc.

A list of the 9 implicit objects is given below:

Object	Type
out	JspWriter
request	HttpServletRequest
response	HttpServletResponse
config	ServletConfig
application	ServletContext
session	HttpSession
pageContext	PageContext
page	Object
exception	Throwable

1) JSP out implicit object

For writing any data to the buffer, JSP provides an implicit object named out. It is the object of JspWriter. In case of servlet you need to write:

1. `PrintWriter out=response.getWriter();`

But in JSP, you don't need to write this code.

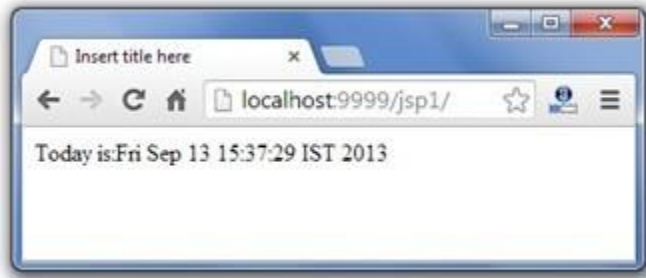
Example of out implicit object

In this example we are simply displaying date and time.

index.jsp

1. `<html>`
2. `<body>`
3. `<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>`
4. `</body>`
5. `</html>`

Output



javatpoint.com

JSP request implicit object

The **JSP request** is an implicit object of type `HttpServletRequest` i.e. created for each jsp request by the web container. It can be used to get request information such as parameter, header information, remote address, server name, server port, content type, character encoding etc.

It can also be used to set, get and remove attributes from the jsp request scope.

Let's see the simple example of request implicit object where we are printing the name of the user with welcome message.

Example of JSP request implicit object

index.html

1. `<form action="welcome.jsp">`

2. `<input type="text" name="uname">`
3. `<input type="submit" value="go"><br/>`
4. `</form>`

welcome.jsp

1. `<%`
2. `String name=request.getParameter("uname");`
3. `out.print("welcome "+name);`
4. `%>`

Output



3) JSP response implicit object

In JSP, response is an implicit object of type `HttpServletResponse`. The instance of `HttpServletResponse` is created by the web container for each jsp request.

It can be used to add or manipulate response such as redirect response to another resource, send error etc.

Let's see the example of response implicit object where we are redirecting the response to the Google.

Example of response implicit object

index.html

1. `<form action="welcome.jsp">`
2. `<input type="text" name="uname">`
3. `<input type="submit" value="go"><br/>`
4. `</form>`

welcome.jsp

1. `<%`
2. `response.sendRedirect("http://www.google.com");`
3. `%>`

4) JSP config implicit object

In JSP, config is an implicit object of type *ServletConfig*. This object can be used to get initialization parameter for a particular JSP page. The config object is created by the web container for each jsp page.

Generally, it is used to get initialization parameter from the web.xml file.

Example of config implicit object:

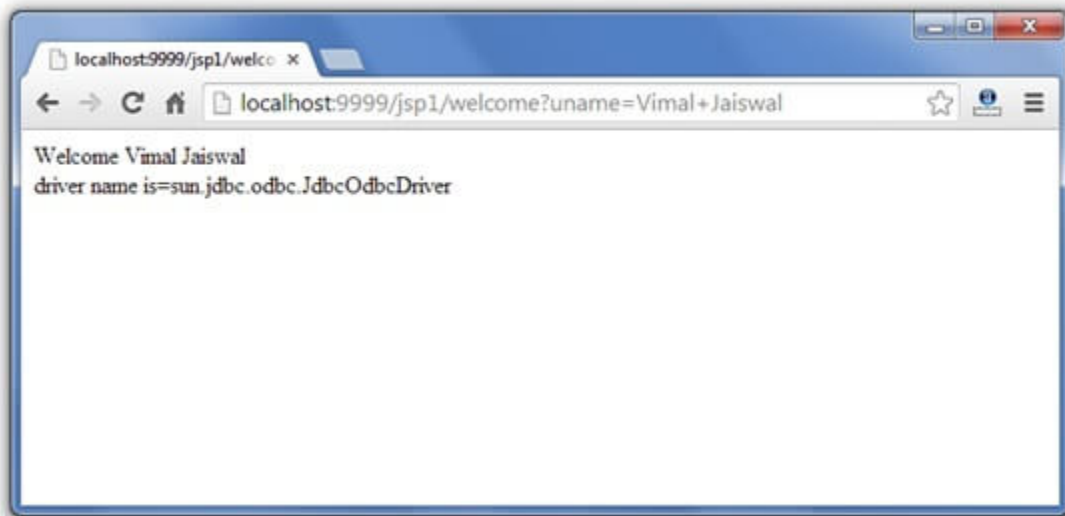
index.html

1. `<form action="welcome">`
2. `<input type="text" name="uname">`
3. `<input type="submit" value="go"><br/>`
4. `</form>`

welcome.jsp

1. `<%`
2. `out.print("Welcome "+request.getParameter("uname"));`

- 3.
4. `String driver=config.getInitParameter("dname");`
5. `out.print("driver name is="+driver);`
6. `%>`



5) JSP application implicit object

In JSP, application is an implicit object of type *ServletContext*.

The instance of *ServletContext* is created only once by the web container when application or project is deployed on the server.

This object can be used to get initialization parameter from configuration file (web.xml). It can also be used to get, set or remove attribute from the application scope.

This initialization parameter can be used by all jsp pages.

Example of application implicit object:

index.html

1. `<form action="welcome">`
2. `<input type="text" name="uname">`

3. `<input type="submit" value="go"><br/>`
4. `</form>`

welcome.jsp

1. `<%`
- 2.
3. `out.print("Welcome "+request.getParameter("uname"));`
- 4.
5. `String driver=application.getInitParameter("dname");`
6. `out.print("driver name is="+driver);`
- 7.
8. `%>`

Output

6) session implicit object

In JSP, session is an implicit object of type HttpSession. The Java developer can use this object to set, get or remove attribute or to get session information.

Example of session implicit object

index.html

1. `<html>`
2. `<body>`
3. `<form action="welcome.jsp">`
4. `<input type="text" name="uname">`
5. `<input type="submit" value="go"><br/>`
6. `</form>`

7. </body>
8. </html>

welcome.jsp

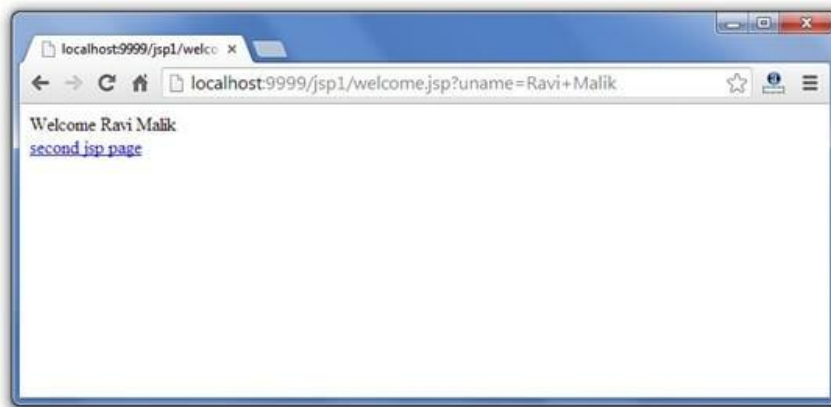
1. <html>
2. <body>
3. <%
- 4.
5. String name=request.getParameter("uname");
6. out.print("Welcome "+name);
- 7.
8. session.setAttribute("user",name);
- 9.
10. second jsp page
- 11.
12. %>
13. </body>
14. </html>

second.jsp

1. <html>
2. <body>
3. <%
- 4.
5. String name=(String)session.getAttribute("user");
6. out.print("Hello "+name);
- 7.
8. %>
9. </body>

10.</html>

Output



7) pageContext implicit object

In JSP, pageContext is an implicit object of type PageContext class. The pageContext object can be used to set, get or remove attribute from one of the following scopes:

- page
- request
- session
- application

In JSP, page scope is the default scope.

Example of pageContext implicit object

index.html

1. <html>
2. <body>
3. <form action="welcome.jsp">
4. <input type="text" name="uname">
5. <input type="submit" value="go">

6. </form>
7. </body>
8. </html>

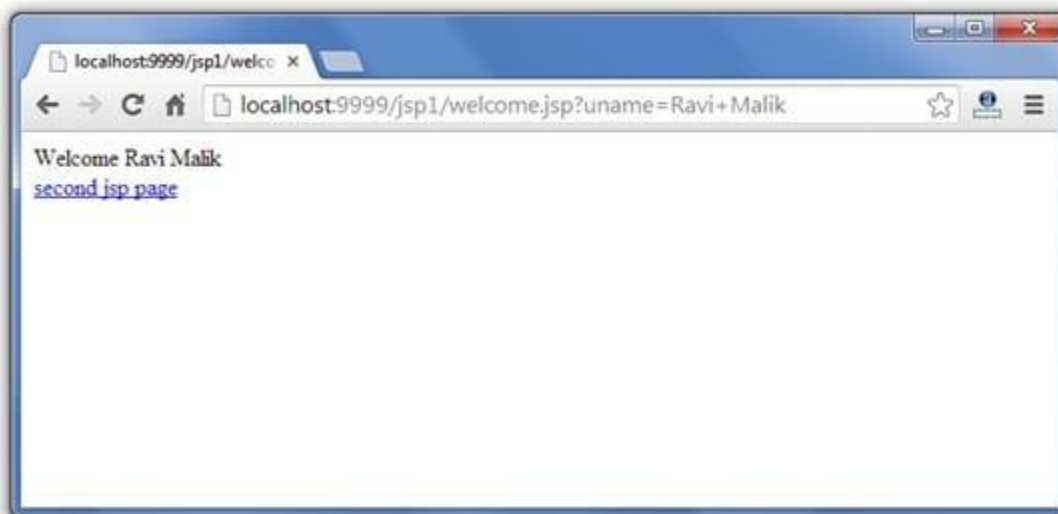
welcome.jsp

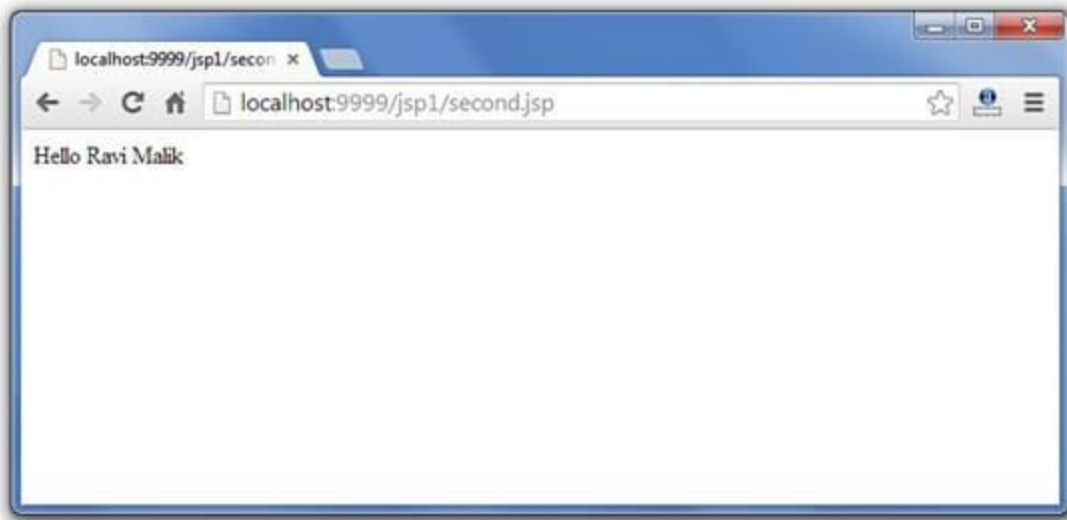
1. <html>
2. <body>
3. <%
- 4.
5. String name=request.getParameter("uname");
6. out.print("Welcome "+name);
- 7.
8. pageContext.setAttribute("user",name,PageContext.SESSION_SCOPE);
- 9.
10. second jsp page
- 11.
12. %>
13. </body>
14. </html>

second.jsp

1. <html>
2. <body>
3. <%
- 4.
5. String
 name=(String)pageContext.getAttribute("user",PageContext.SESSION_SCOPE);
6. out.print("Hello "+name);
- 7.
8. %>
9. </body>
- 10.</html>

Output





8) page implicit object:

In JSP, page is an implicit object of type Object class. This object is assigned to the reference of auto generated servlet class. It is written as:

```
Object page=this;
```

For using this object it must be cast to Servlet type. For example:

```
<% (HttpServletRequest)page.log("message"); %>
```

Since, it is of type Object it is less used because you can use this object directly in jsp. For example:

```
<% this.log("message"); %>
```

9) exception implicit object

In JSP, exception is an implicit object of type `java.lang.Throwable` class. This object can be used to print the exception. But it can only be used in error pages. It is better to learn it after page directive. Let's see a simple example:

Example of exception implicit object:

error.jsp

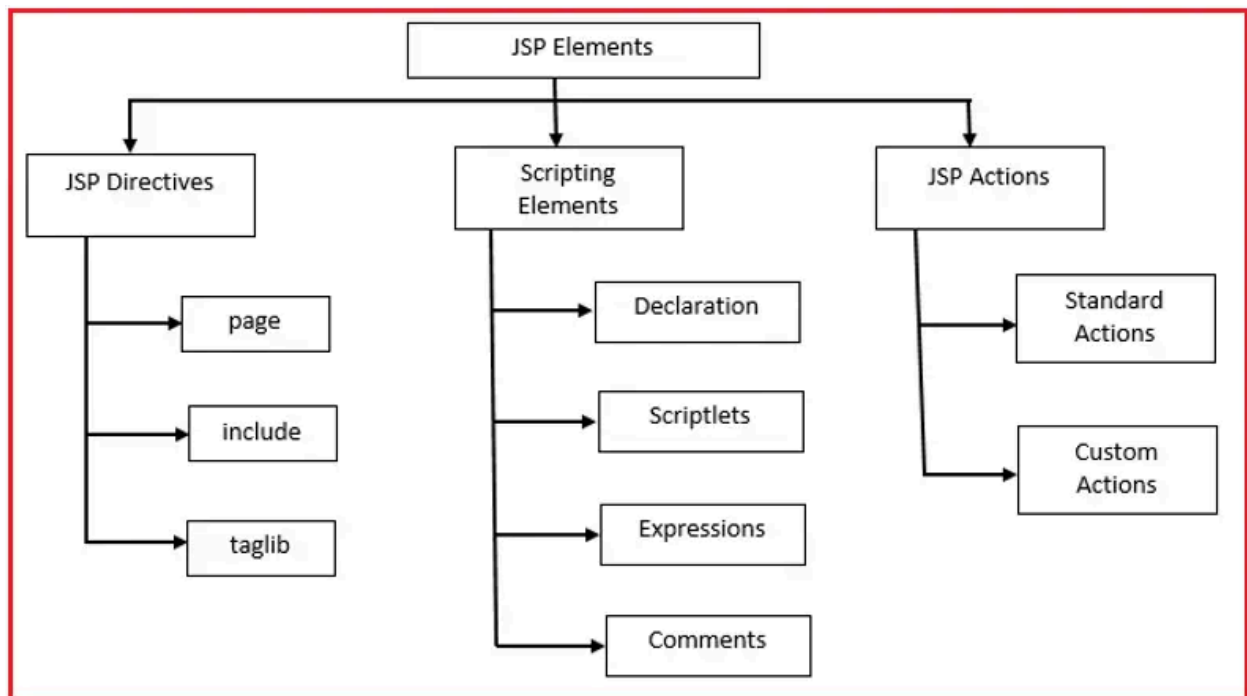
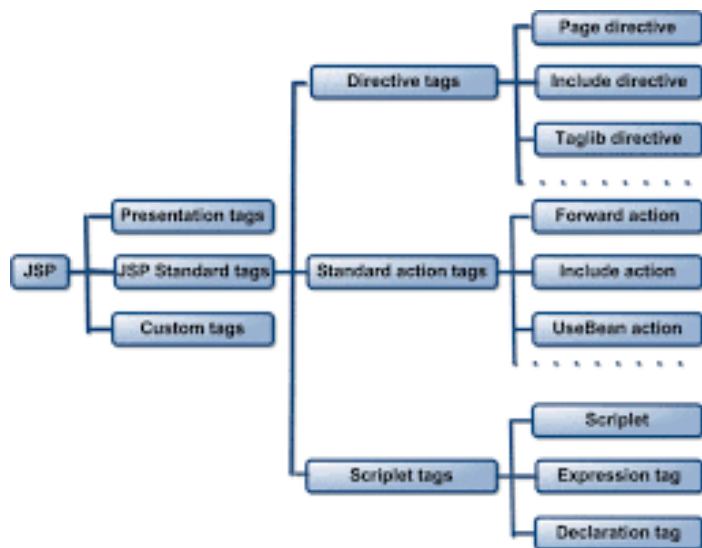
1. `<%@ page isErrorPage="true" %>`
2. `<html>`
3. `<body>`
- 4.
5. Sorry following exception occurred: `<%= exception %>`
- 6.
7. `</body>`
8. `</html>`

JSP directives

The **jsp directives** are messages that tell the web container how to translate a JSP page into the corresponding servlet.

There are three types of directives:

- page directive
- include directive
- taglib directive



Syntax of JSP Directive

1. `<%@ directive attribute="value" %>`

JSP page directive

The page directive defines attributes that apply to an entire JSP page.

Syntax of JSP page directive

1. `<%@ page attribute="value" %>`

Attributes of JSP page directive

- import
- contentType
- extends
- info
- buffer
- language
- isELIgnored
- isThreadSafe
- autoFlush
- session
- pageEncoding
- errorPage
- isErrorPage

1)import

The import attribute is used to import class,interface or all the members of a package.It is similar to import keyword in java class or interface.

Example of import attribute

1. <html>
 2. <body>
 - 3.
 4. <%@ page **import**="java.util.Date" %>
 5. Today is: <%= **new** Date() %>
 - 6.
 7. </body>
 8. </html>
-

2)contentType

The contentType attribute defines the MIME(Multipurpose Internet Mail Extension) type of the HTTP response.The default value is "text/html;charset=ISO-8859-1".

Example of contentType attribute

1. <html>
 2. <body>
 - 3.
 4. <%@ page contentType=application/msword %>
 5. Today is: <%= **new** java.util.Date() %>
 - 6.
 7. </body>
 8. </html>
-

3)extends

The extends attribute defines the parent class that will be inherited by the generated servlet.It is rarely used.

4)info

This attribute simply sets the information of the JSP page which is retrieved later by using `getServletInfo()` method of Servlet interface.

Example of info attribute

1. `<html>`
2. `<body>`
- 3.
4. `<%@ page info="composed by Sonoo Jaiswal" %>`
5. Today is: `<%= new java.util.Date() %>`
- 6.
7. `</body>`
8. `</html>`

The web container will create a method `getServletInfo()` in the resulting servlet. For example:

1. `public String getServletInfo() {`
2. `return "composed by Sonoo Jaiswal";`
3. `}`

5)buffer

The buffer attribute sets the buffer size in kilobytes to handle output generated by the JSP page. The default size of the buffer is 8Kb.

Example of buffer attribute

1. `<html>`
2. `<body>`

- 3.
 4. `<%@ page buffer="16kb" %>`
 5. Today is: `<%= new java.util.Date() %>`
 - 6.
 7. `</body>`
 8. `</html>`
-

6)language

The language attribute specifies the scripting language used in the JSP page. The default value is "java".

7)isELIgnored

We can ignore the Expression Language (EL) in jsp by the isELIgnored attribute. By default its value is false i.e. Expression Language is enabled by default. We see Expression Language later.

1. `<%@ page isELIgnored="true" %> //Now EL will be ignored`
-

8)isThreadSafe

Servlet and JSP both are multithreaded. If you want to control this behaviour of JSP page, you can use isThreadSafe attribute of page directive. The value of isThreadSafe value is true. If you make it false, the web container will serialize the multiple requests, i.e. it will wait until the JSP finishes responding to a request before passing another request to it. If you make the value of isThreadSafe attribute like:

```
<%@ page isThreadSafe="false" %>
```

The web container in such a case, will generate the servlet as:

1. **public class** SimplePage_jsp **extends** HttpJspBase
 2. **implements** SingleThreadModel{
 3.
 4. }
-

9)errorPage

The errorPage attribute is used to define the error page, if exception occurs in the current page, it will be redirected to the error page.

Example of errorPage attribute

1. `//index.jsp`
 2. `<html>`
 3. `<body>`
 - 4.
 5. `<%@ page errorPage="myerrorpage.jsp" %>`
 - 6.
 7. `<%= 100/0 %>`
 - 8.
 9. `</body>`
 10. `</html>`
-

10)isErrorPage

The isErrorPage attribute is used to declare that the current page is the error page

Example of isErrorPage attribute

1. `//myerrorpage.jsp`
2. `<html>`
3. `<body>`
- 4.
5. `<%@ page isErrorPage="true" %>`
- 6.
7. `Sorry an exception occurred!<br/>`
8. `The exception is: <%= exception %>`
- 9.
10. `</body>`
11. `</html>`

Jsp Include Directive

The include directive is used to include the contents of any resource it may be jsp file, html file or text file. The include directive includes the original content of the included resource at page translation time (the jsp page is translated only once so it will be better to include static resource).

Advantage of Include directive

Code Reusability

Syntax of include directive

1. `<%@ include file="resourceName" %>`

Example of include directive

In this example, we are including the content of the header.html file. To run this example you must create an header.html file.

1. `<html>`

2. <body>
- 3.
4. <%@ include file="header.html" %>
- 5.
6. Today is: <%= java.util.Calendar.getInstance().getTime() %>
- 7.
8. </body>
9. </html>
- 10.

JSP Taglib directive

The JSP taglib directive is used to define a tag library that defines many tags. We use the TLD (Tag Library Descriptor) file to define the tags. In the custom tag section we will use this tag so it will be better to learn it in custom tag.

Syntax JSP Taglib directive

1. <%@ taglib uri="uriofthetaglibrary" prefix="prefixoftaglibrary" %>
-

Example of JSP Taglib directive

In this example, we are using our tag named currentDate. To use this tag we must specify the taglib directive so the container may get information about the tag.

1. <html>
2. <body>
- 3.
4. <%@ taglib uri="http://www.javatpoint.com/tags" prefix="mytag" %>
- 5.
6. <mytag:currentDate/>
- 7.

8. </body>
9. </html>
- 10.

JSP Action Tags

There are many JSP action tags or elements. Each JSP action tag is used to perform some specific tasks.

The action tags are used to control the flow between pages and to use Java Bean. The Jsp action tags are given below.

JSP Action Tags	Description
jsp:forward	forwards the request and response to another resource.
jsp:include	includes another resource.
jsp:useBean	creates or locates bean object.
jsp:setProperty	sets the value of property in bean object.
jsp:getProperty	prints the value of property of the bean.
jsp:plugin	embeds another components such as applet.
jsp:param	sets the parameter value. It is used in forward and include mostly.
jsp:fallback	can be used to print the message if plugin is working. It is used in jsp:plugin.

The `jsp:useBean`, `jsp:setProperty` and `jsp:getProperty` tags are used for bean development. So we will see these tags in bean development.

jsp:forward action tag

The `jsp:forward` action tag is used to forward the request to another resource it may be jsp, html or another resource.

Syntax of `jsp:forward` action tag without parameter

1. `<jsp:forward page="relativeURL | <%= expression %>" />`

Syntax of `jsp:forward` action tag with parameter

1. `<jsp:forward page="relativeURL | <%= expression %>">`
2. `<jsp:param name="parametername" value="parametervalue | <%=expression%>" />`
3. `</jsp:forward>`

Example of `jsp:forward` action tag without parameter

In this example, we are simply forwarding the request to the `printdate.jsp` file.

`index.jsp`

1. `<html>`
2. `<body>`
3. `<h2>this is index page</h2>`
- 4.
5. `<jsp:forward page="printdate.jsp" />`
6. `</body>`
7. `</html>`

printdate.jsp

1. <html>
2. <body>
3. <% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
4. </body>
5. </html>

Example of jsp:forward action tag with parameter

In this example, we are forwarding the request to the printdate.jsp file with parameter and printdate.jsp file prints the parameter value with date and time.

index.jsp

1. <html>
2. <body>
3. <h2>**this** is index page</h2>
- 4.
5. <jsp:forward page="printdate.jsp" >
6. <jsp:param name="name" value="javatpoint.com" />
7. </jsp:forward>
- 8.
9. </body>
10. </html>

printdate.jsp

1. <html>
2. <body>
- 3.

4. `<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>`
5. `<%= request.getParameter("name") %>`
- 6.
7. `</body>`
8. `</html>`
- 9.

JSTL (JSP Standard Tag Library)

The JSP Standard Tag Library (JSTL) represents a set of tags to simplify the JSP development.

Advantage of JSTL

1. **Fast Development** JSTL provides many tags that simplify the JSP.
2. **Code Reusability** We can use the JSTL tags on various pages.
3. **No need to use scriptlet tag** It avoids the use of scriptlet tag.

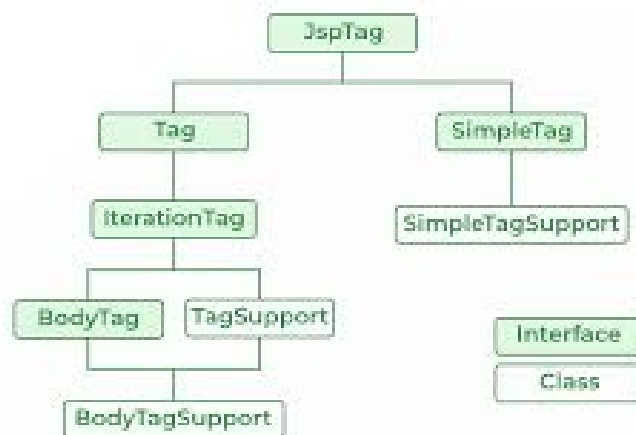
JSTL Tags

There JSTL mainly provides five types of tags:

Tag Name	Description
Core tags	The JSTL core tag provide variable support, URL management, flow control, etc. The URL for the core tag is http://java.sun.com/jsp/jstl/core . The prefix of core tag is c .

Function tags	The functions tags provide support for string manipulation and string length. The URL for the functions tags is http://java.sun.com/jsp/jstl/functions and prefix is fn .
Formatting tags	The Formatting tags provide support for message formatting, number and date formatting, etc. The URL for the Formatting tags is http://java.sun.com/jsp/jstl/fmt and prefix is fmt .
XML tags	The XML tags provide flow control, transformation, etc. The URL for the XML tags is http://java.sun.com/jsp/jstl/xml and prefix is x .
SQL tags	The JSTL SQL tags provide SQL support. The URL for the SQL tags is http://java.sun.com/jsp/jstl/sql and prefix is sql .

Custom Tags in JSP



Custom tags are user-defined tags. They eliminate the possibility of scriptlet tag and separate the business logic from the JSP page.

The same business logic can be used many times by the use of custom tag.

Advantages of Custom Tags

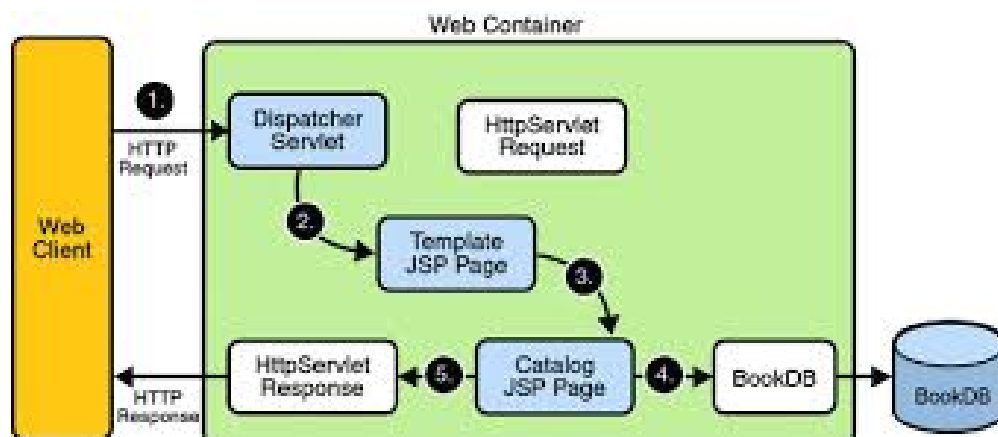
The key advantages of Custom tags are as follows:

1. **Eliminates the need of scriptlet tag** The custom tags eliminates the need of scriptlet tag which is considered bad programming approach in JSP.
2. **Separation of business logic from JSP** The custom tags separate the the business logic from the JSP page so that it may be easy to maintain.
3. **Re-usability** The custom tags makes the possibility to reuse the same business logic again and again.

Syntax to use custom tag

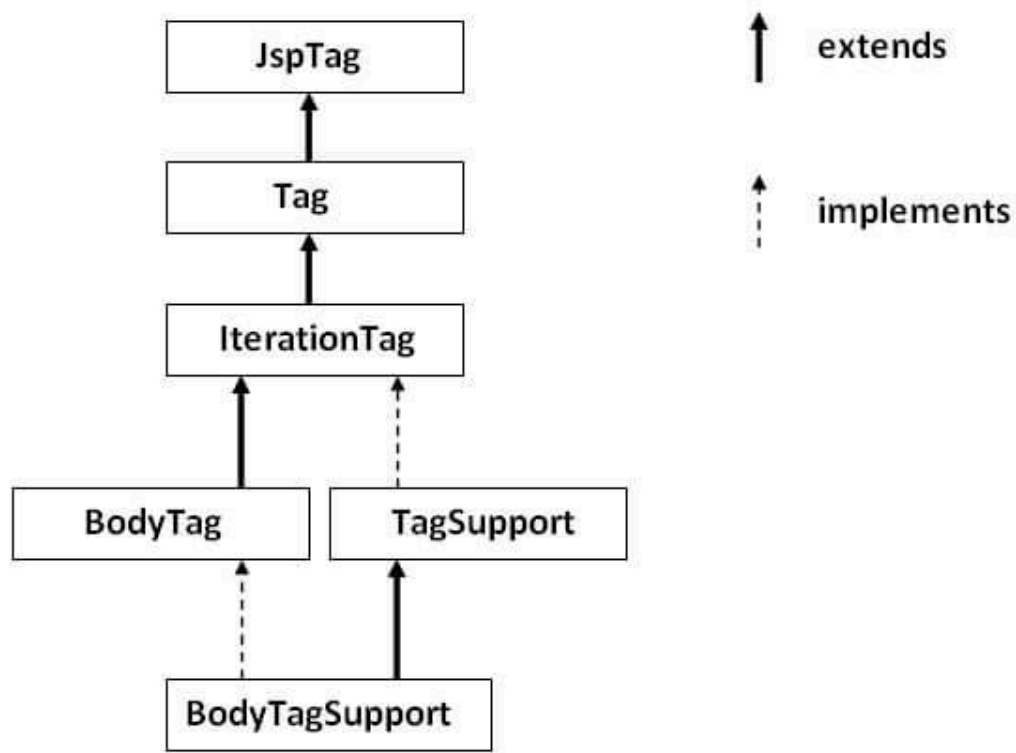
There are two ways to use the custom tag. They are given below:

1. `<prefix:tagname attr1=value1....attrn=valuen />`
1. `<prefix:tagname attr1=value1....attrn=valuen >`
2. body code
3. `</prefix:tagname>`



JSP Custom Tag API

The `javax.servlet.jsp.tagext` package contains classes and interfaces for JSP custom tag API. The `JspTag` is the root interface in the Custom Tag hierarchy.



JspTag interface

The JspTag is the root interface for all the interfaces and classes used in custom tag. It is a marker interface.

Tag interface

The Tag interface is the sub interface of JspTag interface. It provides methods to perform action at the start and end of the tag.

Fields of Tag interface

There are four fields defined in the Tag interface. They are:

Field Name	Description
public static int EVAL_BODY_INCLUDE	it evaluates the body content.
public static int EVAL_PAGE	it evaluates the JSP page content after the custom tag.
public static int SKIP_BODY	it skips the body content of the tag.
public static int SKIP_PAGE	it skips the JSP page content after the custom tag.

Methods of Tag interface

The methods of the Tag interface are as follows:

Method Name	Description
public void setPageContext(PageContext pc)	it sets the given PageContext object.
public void setParent(Tag t)	it sets the parent of the tag handler.
public Tag getParent()	it returns the parent tag handler object.
public int doStartTag()throws JspException	it is invoked by the JSP page implementation object. The JSP programmer should override this method and define the business logic to be performed at the start of the tag.

public int doEndTag()throws JspException	it is invoked by the JSP page implementation object. The JSP programmer should override this method and define the business logic to be performed at the end of the tag.
public void release()	it is invoked by the JSP page implementation object to release the state.

IterationTag interface

The IterationTag interface is the sub interface of the Tag interface. It provides an additional method to reevaluate the body.

Field of IterationTag interface

There is only one field defined in the IterationTag interface.

- **public static int EVAL_BODY_AGAIN** it reevaluates the body content.

Method of Tag interface

There is only one method defined in the IterationTag interface.

- **public int doAfterBody()throws JspException** it is invoked by the JSP page implementation object after the evaluation of the body. If this method returns EVAL_BODY_INCLUDE, body content will be reevaluated, if it returns SKIP_BODY, no more body content will be evaluated.
-

TagSupport class

The TagSupport class implements the IterationTag interface. It acts as the base class for new Tag Handlers. It provides some additional methods also.

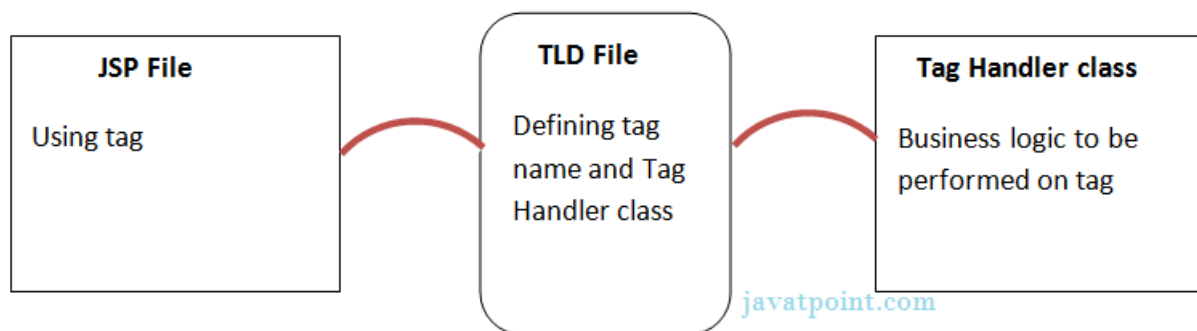
Example of JSP Custom Tag

In this example, we are going to create a **custom tag that prints the current date and time**. We are performing action at the start of tag.

For creating any custom tag, we need to follow following steps:

1. **Create the Tag handler class** and perform action at the start or at the end of the tag.
2. **Create the Tag Library Descriptor (TLD) file** and define tags
3. **Create the JSP file that uses the Custom tag defined in the TLD file**

Understanding flow of custom tag in jsp



1) Create the Tag handler class

To create the Tag Handler, we are inheriting the **TagSupport class** and overriding its method **doStartTag()**. To write data for the jsp, we need to use the **JspWriter class**.

The **PageContext** class provides **getOut()** method that returns the instance of JspWriter class. TagSupport class provides instance of pageContext by default.

File: MyTagHandler.java

1. **package** com.javatpoint.sonoo;
2. **import** java.util.Calendar;
3. **import** javax.servlet.jsp.JspException;
4. **import** javax.servlet.jsp.JspWriter;
5. **import** javax.servlet.jsp.tagext.TagSupport;
6. **public class** MyTagHandler **extends** TagSupport{
- 7.
8. **public int** doStartTag() **throws** JspException {
9. JspWriter out=pageContext.getOut();//returns the instance of JspWriter
10. **try**{
11. out.print(Calendar.getInstance().getTime());//printing date and time using
 JspWriter
12. }**catch**(Exception e){System.out.println(e);}
13. **return** SKIP_BODY;//will not evaluate the body content of the tag
- 14.}
- 15.}

2) Create the TLD file

Tag Library Descriptor (TLD) file contains information of tag and Tag Handler classes. It must be contained inside the **WEB-INF** directory.

File: mytags.tld

1. **<?xml** **version**="1.0" **encoding**="ISO-8859-1" **?>**
2. **<!DOCTYPE** taglib
3. **PUBLIC** "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
4. **"http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">**
- 5.
6. **<taglib>**
- 7.

8. `<tlib-version>1.0</tlib-version>`
9. `<jsp-version>1.2</jsp-version>`
10. `<short-name>simple</short-name>`
11. `<uri>http://tomcat.apache.org/example-taglib</uri>`
- 12.
13. `<tag>`
14. `<name>today</name>`
15. `<tag-class>com.javatpoint.sonoo.MyTagHandler</tag-class>`
16. `</tag>`
17. `</taglib>`

3) Create the JSP file

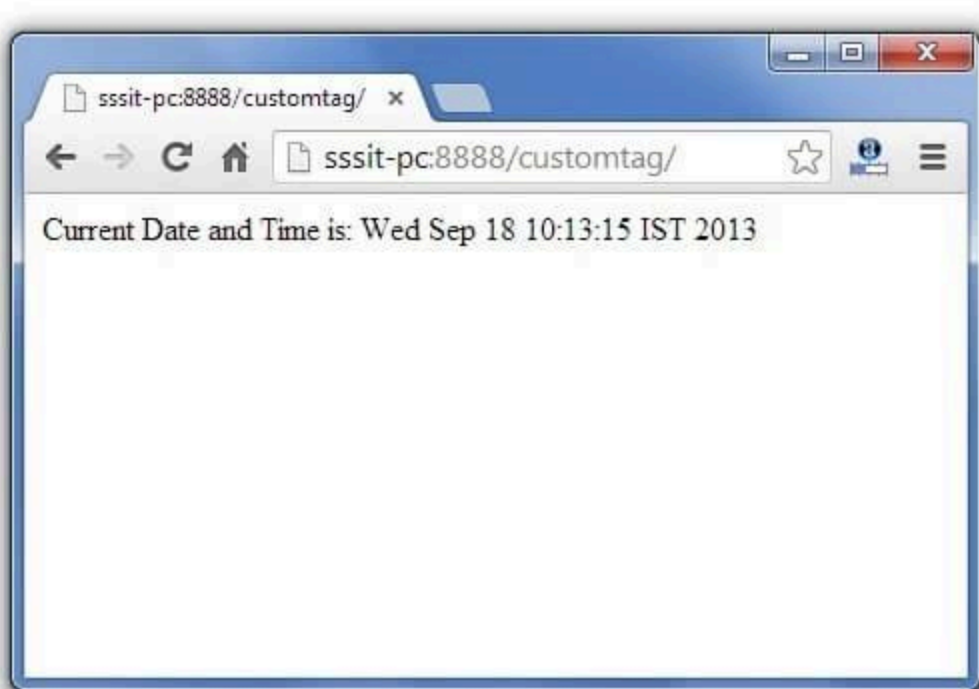
Let's use the tag in our jsp file. Here, we are specifying the path of tld file directly. But it is recommended to use the uri name instead of full path of tld file. We will learn about uri later.

It uses **taglib** directive to use the tags defined in the tld file.

File: index.jsp

1. `<%@ taglib uri="WEB-INF/mytags.tld" prefix="m" %>`
2. `Current Date and Time is: <m:today/>`

Output



Attributes in JSP Custom Tag

Attributes in JSP Custom Tag

Example to use attribute in JSP Custom Tag

There can be defined too many attributes for any custom tag. To define the attribute, you need to perform two tasks:

- Define the property in the TagHandler class with the attribute name and define the setter method
- define the attribute element inside the tag element in the TLD file

Let's understand the attribute by the tag given below:

1. `<m:cube number="4"></m:cube>`

Here **m** is the prefix, **cube** is the tag name and **number** is the attribute.

Simple example of attribute in JSP Custom Tag

In this example, we are going to use the cube tag which return the cube of any given number. Here, we are defining the number attribute for the cube tag. We are using the three file here:

- index.jsp
- CubeNumber.java
- mytags.tld

index.jsp

1. `<%@ taglib uri="WEB-INF/mytags.tld" prefix="m" %>`
2. Cube of 4 is: `<m:cube number="4"></m:cube>`

CubeNumber.java

1. `package com.javatpoint.taghandler;`
2. `import javax.servlet.jsp.JspException;`
3. `import javax.servlet.jsp.JspWriter;`
4. `import javax.servlet.jsp.tagext.TagSupport;`
- 5.
6. `public class CubeNumber extends TagSupport{`
7. `private int number;`
- 8.
9. `public void setNumber(int number) {`
10. `this.number = number;`
11. `}`
- 12.
13. `public int doStartTag() throws JspException {`
14. `JspWriter out=pageContext.getOut();`
15. `try{`


```
16.     out.print(number*number*number);
17. }catch(Exception e){e.printStackTrace();}
18.
19.     return SKIP_BODY;
20.}
21.}
```

mytags.tld

```
1. <?xml version="1.0" encoding="ISO-8859-1" ?>
2. <!DOCTYPE taglib
3.     PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
4.     "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
5.
6. <taglib>
7.     <tlib-version>1.0</tlib-version>
8.     <jsp-version>1.2</jsp-version>
9.     <short-name>simple</short-name>
10.    <uri>http://tomcat.apache.org/example-taglib</uri>
11.    <description>A simple tag library for the examples</description>
12.
13.    <tag>
14.        <name>cube</name>
15.        <tag-class>com.javatpoint.taghandler.CubeNumber</tag-class>
16.        <attribute>
17.            <name>number</name>
18.            <required>true</required>
19.        </attribute>
20.    </tag>
21.</taglib>
```

Output

1. Cube of 4 is: 64

Custom URI in JSP Custom Tag

We can use the custom URI, to tell the web container about the tld file. In such case, we need to define the taglib element in the web.xml. The web container gets the information about the tld file from the web.xml file for the specified URI.

Example to use custom URI in JSP Custom Tag

In this example, we are going to use the custom uri in the JSP file. For this application, we need to focus on 4 files.

- index.jsp
- web.xml
- mytags.tld
- PrintDate.java

index.jsp

1. `<%@ taglib uri="mytags" prefix="m" %>`
2. Today is: `<m:today></m:today>`

web.xml

1. `<?xml version="1.0" encoding="UTF-8"?>`
2. `<!DOCTYPE web-app`
3. `PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"`
4. `"http://java.sun.com/dtd/web-app_2_3.dtd">`
- 5.
6. `<web-app>`

- 7.
 8. <jsp-config>
 9. <taglib>
 10. <taglib-uri>mytags</taglib-uri>
 11. <taglib-location>/WEB-INF/mytags.tld</taglib-location>
 12. </taglib>
 13. </jsp-config>
 - 14.
 15. </web-app>
-

mytags.tld

1. <?xml version="1.0" encoding="ISO-8859-1" ?>
2. <!DOCTYPE taglib
3. PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
4. "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
- 5.
6. <taglib>
7. <tlib-version>1.0</tlib-version>
8. <jsp-version>1.2</jsp-version>
9. <**short**-name>simple</**short**-name>
10. <uri>mytags</uri>
11. <description>A simple tag library **for** the examples</description>
- 12.
13. <tag>
14. <name>today</name>
15. <tag-**class**>com.javatpoint.taghandler.PrintDate</tag-**class**>
16. </tag>
17. </taglib>

